

SLO-Aware Scheduling for Large Language Model Inferences

Jinqi Huang
Huawei Technologies Co., Ltd
huangjinqi1@huawei.com

Wenjie Huang
Huawei Technologies Co., Ltd
huangwenjie16@huawei.com

Yi Xiong
Huawei Technologies Co., Ltd
xiongyi26@huawei.com

Entong Li
Huawei Technologies Co., Ltd
lientong@huawei.com

Xin Chen
Huawei Technologies Co., Ltd
chenxin@huawei.com

Xuebing Yu
Huawei Technologies Co., Ltd
yuxuebing3@huawei.com

Li Zeng
Huawei Technologies Co., Ltd
zengli43@huawei.com

ABSTRACT

Large language models (LLMs) have revolutionized applications such as code completion, chatbots, and online classification. To elevate user experiences, service level objectives (SLOs) serve as crucial benchmarks for assessing inference services capabilities. In practice, an inference service processes multiple types of tasks, each with its own distinct SLO. To ensure satisfactory user experiences, each request’s distinct SLOs should be considered in scheduling. However, existing designs lack this consideration, leading to insufficient hardware utility and suboptimal performance.

This paper analyzes scenarios to process tasks with varying SLOs, and introduces a simulated annealing-based scheduler to decide request priority sequence based on a request’s SLO, input lengths, and possible output lengths. As the first specialized scheduler for multi-SLO scenarios, this work improves SLO attainment by up to 5× and reduces average latency by 31.6% on Python-Code-23k-ShareGPT and ShareGPT_Vicuna_unfiltered datasets, compared to current state-of-the-art framework vLLM and a new framework LMDeploy.

1 INTRODUCTION

The rapid revolution of LLMs has been witnessed since the debut of ChatGPT [2], with a succession of LLMs being unveiled, including Llama [22] by Meta, Gemini [30] by Google, and Qwen [27] by Alibaba. Since then, LLMs have gained extensive adoption across a multitude of industry applications such as fault maintenance, customer service, recommendation system, code completion [4, 31], question answering [3, 17], knowledge extraction [8, 10], document summarization [21, 33], search engines [6, 20], etc.

LLMs are integrated into inference serving systems to provide services. These systems, such as vLLM [18], fastTransformer [23], and Triton [24], queue, schedule, and batch the incoming queries before feeding them into LLMs. SLOs are introduced to evaluate if a LLM inference service deliver a satisfactory user experience due to LLM services’ interactive nature. As the result of diverse user experiences across tasks, SLOs typically vary by applications. For an instance, offline processing emphasizes throughput over latency to reduce costs, while online classifiers require real-time responses, making the end-to-end (e2e) latency critical. For chatbots, focusing on time-to-first-token (TTFT) and time-per-output-token (TPOT)

ensures quick initial responses and subsequent token generation with human-like reading speed.

In practical industry applications (e.g., fault maintenance, customer service, or recommendation system), a set of LLM inference services are employed for various tasks, each with its own SLO. Figures A and B in 1 show examples of varying SLO requirements for different tasks. In Scenario 1, as depicted in Figure 1(A), an application may involve multiple subtasks, each with varying input and output lengths, request concurrency and specific SLOs. Similarly, as shown in Figure 1(B), Scenario 2 presents a case where multiple applications with diverse SLOs utilize a shared resource pool, which encompasses more than one LLM inference service. Figure 1(C) gives an example of multiple SLOs introduced by Scenario 1 or 2. To fully utilize the hardware resources and to fulfill application SLOs as much as possible, it is important to account for the varying SLOs across different applications.

Mainstream inference services, including vLLMs [18], Triton [24], and fastTransformer [23], focus on high-throughput low-latency scheduling with first-come-first-serve (FCFS) policies. Besides, other high-performance schedulers enhance throughput with specialized techniques: Sarathi-Serve [1] introduces chunked prefill to avoid stalls during transitions between prefill and decode phases; Fast-Serve [37] implements skip-join multi-level feedback queues and iteration-level preemption to mitigate head-of-line blocking caused by longer sequences processed before shorter ones; Splitwise [25] and DistServe [42] isolate prefill and decode phases into distinct inference instances and optimizing them separately; Llmunix [29] leverages load balancing, prioritization, de-fragmentation, and auto-scaling scheduling strategies to achieve high throughput. All these existing designs are tailored to scenarios where queries originate from a single task or from various tasks with identical SLOs. When immigrated to multi-SLO scenarios, these existing systems lack awareness of the unique SLOs associated with different tasks, leading to suboptimal hardware utilization and SLO attainment.

To address the complexity of multi-SLO scheduling, this paper propose a SLO-aware priority mapping algorithm. Aiming at optimizing the system SLO attainment and minimizing the average latency, we optimize each request’s priority and the actual batch size, based on task-specific SLOs, input lengths, and output lengths prediction. To mitigate substantial searching overhead for the optimal priority sequence and request batch sizes, we employ a simulated annealing-based SLO-aware scheduling approach with an

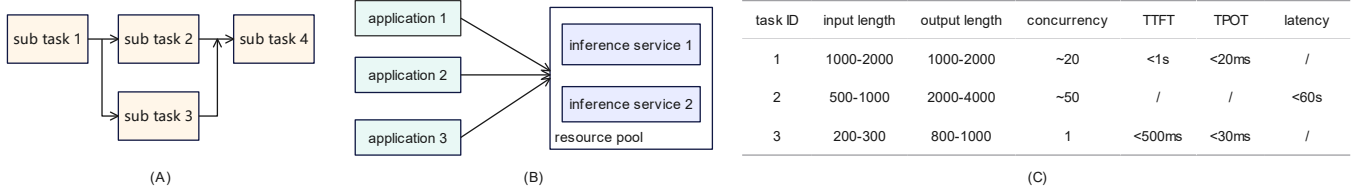


Figure 1: Illustration of multiple SLOs existing in LLM inference: (A) distinct SLOs for multiple sub-tasks within an application; (B) diverse SLOs for applications sharing resources; (C) multiple SLOs introduced by scenarios (A) and (B), with '/' indicates unimportance for this task.

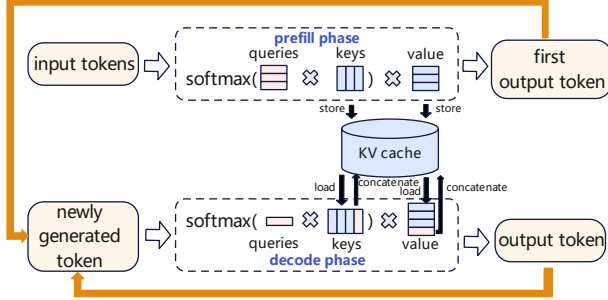


Figure 2: Prefill and decode phases in LLM inferences. KV cache is dynamically stored, loaded, and updated during the inference.

overhead of 1ms, while maintaining a similar effectiveness to an exhaustive search-based method. To the best of our knowledge, this work is the first system designed specifically to address SLO-aware scheduling challenges. To evaluate the system effectiveness, we benchmark the performance against the state-of-the-art framework vLLM and a newly-introduced framework LMDeploy [7] with ShareGPT_Vicuna_unfiltered [32] and Python-Code-23k-ShareGPT [14] datasets, and prove that our system boosts the SLO attainment by up to 5× and reduces the average latency by up to 31.6%, respectively. The contributions of this paper are summarized below:

- We propose a priority mapping algorithm for prioritizing each request based on its SLO, input length, and expected output length if available, to achieve highest SLO attainment with average latency as low as possible.
- We develop the first specialized simulated annealing-based SLO-aware scheduler with low overhead to tackle the complexities of SLO-aware scheduling.
- We demonstrate the scheduling system on Python-Code-23k-ShareGPT and ShareGPT_Vicuna_unfiltered datasets and prove that our method outperforms vLLM and LMDeploy in SLO attainment by up to 5× and reduces average latency by up to 31.6%.

2 BACKGROUND

2.1 Preliminaries

LLMs evolve from transformers [34] with an attention-based architecture. Unlike their deep neural networks (DNNs) predecessors, transformer-based LLMs follow the principle of 'next token prediction' to construct output sequences: LLM inference generates

one token per iteration based on the user prompt and previous tokens. This working principle forms the characteristic of variable and unpredictable output lengths in LLM. Another important feature of LLMs is the significant diversity between prefill and decode phases. As shown in Figure 2, the prefill phase refers to the procedure from receiving a user prompt to generating the first token. During this procedure, tokens from the prompt are processed concurrently to generate a single new token. This procedure involves massive matrix multiplications, given that the attention layers of LLMs encompass three core attention matrices—*Queries*, *Keys* and *Values*—all requiring self-attention computations. During decoding, the prompt is extended with prior tokens to generate the next one. The *Keys* and *Values* from the previous step are saved as *KV cache* for reuse, with only the *Queries* vector computed from the last token, is employed. Therefore, the *KV cache* save computations for decoding from matrix multiplications to matrix-vector products, making data movement the main bottleneck.

2.2 Related Work

To achieve high throughput and low latency, numerous inference serving systems provides request scheduling and batching techniques. FastTransformer [23] utilized request scheduling and static batching technique: requests in the same batch are processed collectively until all have been completed. Orca [39] was pioneering in introducing continuous batching, a strategy that allows new requests to be seamlessly integrated into the space vacated by completed requests without waiting an entire batch to finish. However, due to the inherent differences between the prefill and decode phases, requests in these distinct stages cannot be processed simultaneously within the same batch, causing a stall for prefilling each time when new requests are executed. Sarathi-serve [1] addressed this challenge by dividing the prefill stage into smaller chunks and interleaving them with ongoing decode operations to form a hybrid batch. vLLM [18] is the state-of-the-art inference framework that combines both continuous batching and chunked prefill for better flexibility and performance, and employs PagedAttention to eliminates the memory fragmentation. LMDeploy [7] is a similar newly introduced framework that utilizes model quantization to accelerate LLM inference. All the aforementioned systems adhere to the FCFS policies, treating all requests equally without consideration for their varying SLO requirements.

In contrast, FastServe[37] assigns priority to requests based on their prompt lengths and implements skip-join Multi-Level Feedback Queue scheduling along with iteration-level preemption to

mitigate the head-of-line blocking inherent in the FCFS and run-to-completion approaches. However, FastServe is not specifically tailored for multi-SLO scenarios, as it bases request priorities solely on input lengths. Llumnix [29] is another work that takes the priority of requests into consideration. It highlighted the needs for new request rescheduling goals such as de-fragmentation and priorities. Nonetheless, it fails to provide an executable solution on how to determine the priority of individual requests based on their SLOs. SOLA [13] and Tempo [40] aim to provide SLO-awareness for scheduling. However, multi-SLO scenarios are not considered.

To fill the vacancy of the multi-SLO scheduler without reinventing the wheel, this paper introduces a decoupled scheduler that can seamlessly integrate with prior historical inference serving systems such as vLLM. Other inference serving designs, such as Splitwise [25] and DistServe [42], which focus on maximizing throughput during the prefill and decode phases, as well as solutions like [19] and [36] that aim to expedite inference in distributed systems, are orthogonal to the objectives of this paper.

3 MOTIVATIONS

3.1 Problem Formulation

Traditionally defined goodput is the number of requests that meet SLOs within a time window, as expressed below:

$$\text{Goodput} = \frac{n}{t'} \quad (1)$$

Here, n denotes the number of requests that meet their SLOs, and t' signifies the total latency incurred by processing all requests. This definition frames SLO attainment, the ratio of requests that meet SLOs, as a key metric indicative of user experience. However, t' ignores waiting time's effect on user satisfaction, with total latency often reflecting the final request's e2e latency, despite Ref [11] showing waiting times significantly affect user experience. Moreover, FastServe [37] notes waiting times can be $8\times$ execution time in practice. Furthermore, Ref [35] criticizes using goodput as an optimization target, as it can lead to request abandonment, increasing user dissatisfaction over mere SLO misses. To more accurately capture the essence of user experience, we introduce a novel metric G defined as follows. Our objective in this study is to maximize G :

$$G = \frac{n}{t} \quad (2)$$

Where t denotes the latency summation of all given requests:

$$t = \sum_{i=0}^{N-1} t_{e2e,i} \quad (3)$$

Given a set of job $j_0, j_1, \dots, j_{N-1}$ with N denotes the number of jobs, their execution time and waiting time are represented as $t_{exec}^0, t_{exec}^1, \dots, t_{exec}^{N-1}$ and $t_{wait}^0, t_{wait}^1, \dots, t_{wait}^{N-1}$, respectively. The e2e latency experienced by a user can be formulated as follows:

$$t_{e2e,i} = t_{exec,i} + t_{wait,i} \quad (4)$$

Where i is the job index ranging from 0 to $N-1$. In this paper, we consider two streaming service tasks: one prioritizes e2e latency, seen in code completion; another emphasizes interaction speed via TTFT and TPOT, seen in chatbots. h_i indicates which type of tasks j_i belongs to. Offline applications, where throughput is the primary

optimization goal rather than user experience, are not in the scope of this work.

$$h_i = \begin{cases} 1, & \text{if } j_i \text{ prioritizes e2e latency} \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

Given $t_{SLO-e2e,i}$ denotes the SLO (e2e latency), n can be given by the expression as follows:

$$n = \sum_{i=0}^{N-1} x_i \quad (6)$$

Here, x_i is defined as a flag to indicate where the job has met the SLO:

$$x_i = \begin{cases} 1, & \text{if } h_i = 1 \text{ and } t_{e2e,i} \leq t_{SLO-e2e,i} \\ 1, & \text{if } h_i = 0 \text{ and } t_{TTFT,i} \leq t_{SLO-TTFT,i} \\ & \text{and } t_{TPOT,i} \leq t_{SLO-TPOT,i} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

Where $t_{TTFT,i}$ and $t_{TPOT,i}$ can be expressed as below:

$$t_{TTFT,i} = t_{prefill,i} + t_{wait,i} \quad (8)$$

$$t_{TPOT,i} = t_{decode,i} \quad (9)$$

It is worth noting that altering the job execution order affects both n and t through $t_{wait,i}$. Additionally, it is essential to determine the inference batch size. However, this batch size fluctuates due to the diversity in the lengths of individual requests. Thus, the batch size is expressed as a sequence b_0, b_1, b_{M-1} where M represents the total number of batches required for the iterative processing of requests, subject to the constraint that $\sum_{i=0}^{M-1} b_i = N$. Given that the position of a job j_i within the sequence is p_i , $p_i \in [0, N-1]$, the batch index to which j_i is allocated can be denoted as a_i :

$$a_i = K + 1, \text{ if } \sum_{k=0}^K b_k \leq p_i < \sum_{k=0}^{K+1} b_k \quad (10)$$

Therefore, $t_{wait,i}$ can be expressed as follows:

$$t_{wait,i} = \begin{cases} \sum_{j=0}^{a_i-1} \max_{i=0}^{N-1} (t_{exec,i} \cdot g(i, j)) & \text{if } a_i \leq 1 \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

Where $g(i, j)$ is a function to indicate whether job j_i is assigned to batch j :

$$g(i, j) = \begin{cases} 1, & \text{if } a_i = j \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

Therefore, the primary objective is to identify a permutation of $p_0, p_1, \dots, p_{N-1}$ and the batch index of each request $a_0, a_1, \dots, a_{N-1}$ that maximize G :

$$\text{argmax}_{\{p_0, p_1, \dots, p_{N-1}, a_0, a_1, \dots, a_{N-1}\}} G \quad (13)$$

Notably, G can also be expressed as the ratio of SLO attainment to average latency. Hence, the primary goal of this paper is to maximize SLO attainment while concurrently minimizing average latency to its lowest feasible level. In the following context, we use G to indicate the optimization objective.

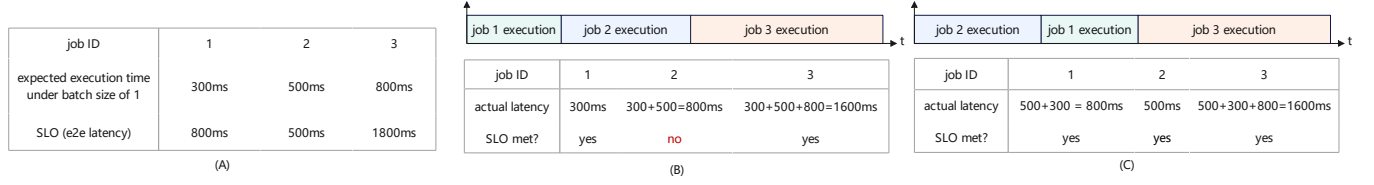


Figure 3: Illustration of an SLO-aware scheduling with the batch size of 1 for simplicity: (A) Three requests with distinct SLOs are received simultaneously, with their SLO is defined by e2e latency. (B) Execution order is based on expected execution time, leading to 2 out of 3 requests meeting SLOs and an accumulated latency of 2700ms, yielding the G of 0.74 req/s. (C) SLO-aware sequencing prioritizes Job 2 over Job 1, with all requests meeting SLOs and an accumulated latency of 2900ms, improving G to 1.03 req/s.

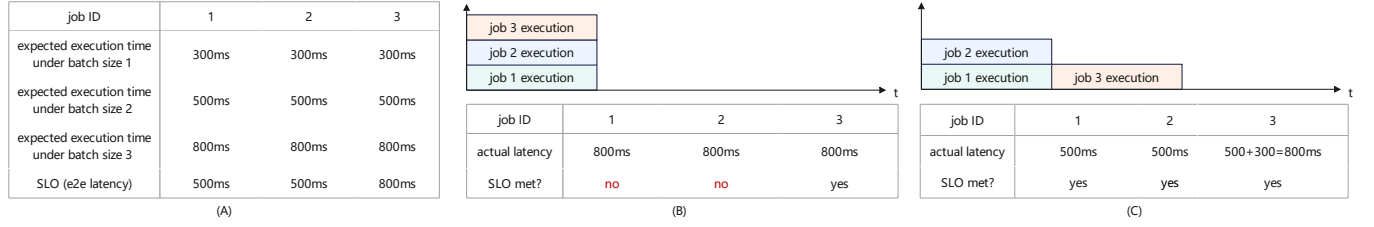


Figure 4: Demonstration of how adjusting the batch size enhances G . With a max batch size of 3: (A) Three requests with different SLOs are received, and execution time increases with batch size. (B) Processing all three requests in one batch results in only 1 meeting SLOs, with an accumulated of 2400ms, giving $G = 0.0004$ req/s. (C) Delaying one request to the next cycle allows all 3 to meet SLOs, with an accumulated latency of 1800ms, increasing G to 0.00167 req/s.

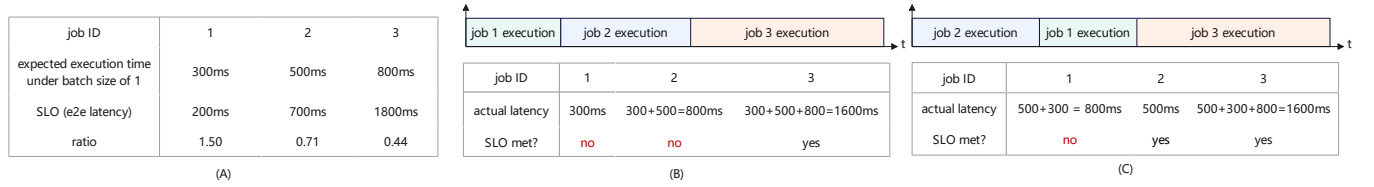


Figure 5: Demonstration of deferring a ‘strict’ SLO request boosting G with the batch size of 1 for simplicity: (A) Three requests using e2e latency as the metric are received, including one with an unachievable SLO. (B) Execution prioritizes based on SLO values and the ratio of the expected latency to the actual SLO, with a higher ratio signifying a tougher SLO to meet. Only 1 out of 3 request meets SLO with an accumulated latency of 2700ms, giving G of 0.37 req/s. (C) Delaying the strict SLO allows others to take precedence, with 2 out of 3 meeting SLOs and an accumulated latency of 2900ms, improving G to 0.74 req/s.

3.2 Opportunity: Priority Scheduling with SLO Awareness

The main issue in current inference systems, as outlined in Section 3.1, is the lack of SLO awareness: existing designs either overlook priority scheduling or not consider the varied SLOs on different request types. Figure 3 illustrates an example of SLO-aware scheduling. When three jobs are received concurrently (shown in Figure 3 (A)), historical scheduling methods focused on reducing wait times based on input size or execution time, ignoring individual request SLOs. The strategy in Figure 3 (B) may not efficiently optimize G for requests with different SLOs. To order requests more effectively, priority scheduling with SLO awareness can be applied: Upon receiving requests, their priority is set based on SLOs and predicted latencies using input lengths and predicted output lengths, then executed accordingly in inference systems. Figure 3 (C) demonstrates

the SLO-aware scheduling approach that yields a superior G . However, integrating ppriority scheduling with SLO awareness into existing inference systems still faces two major challenges.

3.3 Challenge 1: Complexity of SLO-Priority Mapping

SLO-aware scheduling aims to map SLOs and request attributes for processing order. However, constructing such a function is not a simple task. To begin with, various applications require different types of SLOs, including e2e latency, TTFT, TPOT, etc. Meanwhile, different applications exhibit varying degrees of sensitivity to these SLOs. Therefore, it is essential to standardize the expression of primary SLO constraints for each request.

Additionally, the batch size serves as a tunable parameter affecting determination of the priority sequence through its impact on latency. Figure 4 shows the effect of batch size changes on G . ‘Strict’ SLOs for jobs 1 and 2 (Fig. 4A) cause batching all requests to slow down these jobs, despite not exceeding the maximum batch size

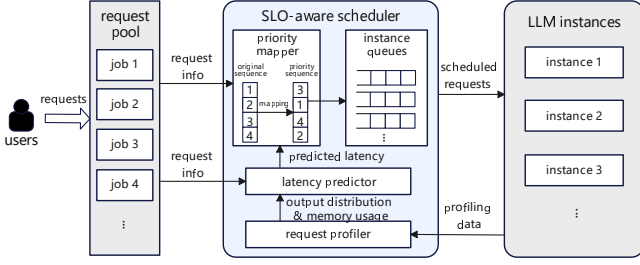


Figure 6: Design overview of the SLO-aware scheduler.

(Fig. 4B). Instead, shifting a ‘loose’ SLO task to the next iteration enhances SLO attainment (Fig. 4C), thus improving G . Accounting for request batching, determining the SLO-aware priority scheduling sequence is further complicated.

Moreover, prioritizing requests solely by SLOs or the difficulty of meeting SLOs does not always optimize G . Occasionally, deferring requests with ‘strict’ SLOs can sometimes improve overall G . By allocating the time saved from these requests, one can accommodate a greater number of requests with more easily achievable SLOs, thereby increasing the total count of requests that meet their SLOs. Figure 5 shows such an example. In Figure 5 (A), job 1 is unable to meet its SLO. Assigning high priority to job 1 results in job 2 also failing to meet its SLO, as depicted in Figure 5 (B). However, by slightly delaying the execution of job 1 (shown in Figure 5 (C)), G is maximized with a cost of slightly increased latency. As a result, these considerations further increases the complexity of the SLO-priority mapping.

3.4 Challenge 2: Inefficient Scheduling Solution Searching

Searching for the optimal priority order is crucial for better performance, but it is also essential to minimize the overhead, as it contributes to the e2e latency. Given n requests, an exhaustive search to find the sequence that maximizes G have a time complexity of $O(n!)$, and traversing over all possible batch size options for each request have a time complexity of $O(2^n)$, resulting in a total time complexity of $O(n! \cdot 2^n)$. As n grows, the search space explodes, making the search time excessively long (over 2 hours for $n > 30$). Consequently, an efficient search algorithm is key for designing an effective SLO-aware scheduler.

4 METHODOLOGY

4.1 Design Overview

Figure 6 presents an architectural overview of the SLO-aware scheduler, which comprises four key components: a request profiler, a latency predictor, a priority mapper, and instance queues. User requests are collected in a request pool, where they await processing. These requests are then fed to the latency predictor and priority mapper for further analysis. The latency predictor combines request information with output distribution and memory usage data provided by the request profiler to predict the latency of each request. Subsequently, the priority mapper leverages the SLOs of the requests along with their predicted latencies to determine the priority order. This priority sequence is then allocated to instance queues, the number of which corresponds to the actual count of

LLM inference instances. Finally, requests are dispatched to these instances for execution.

The implementation details of the request profiler and the latency predictor are elaborated in Section 4.2, and the core algorithm of the priority mapper and the corresponding scheduling strategy are detailed in Sections 4.3 and 4.4, respectively.

4.2 Latency Predictor

The latency predictor aims to provide a reasonable execution time prediction, which can be further used as a criteria to decide the priority sequence. Following the estimation modeling provided by history work [5], the prefill time $t_p(b, l_i)$ increases linearly to both the input length l_i and the batch size b when l_i is below $2k$:

$$t_p(b, l_i) = \alpha_p \cdot b \cdot l_i + \beta_p \cdot b + \gamma_p \cdot l_i + \delta_p \quad (14)$$

Where $\alpha_p, \beta_p, \gamma_p, \delta_p$ denote as fitting coefficients for prefill time. Likewise, the per-token decode time $t_d(b, l)$ is linear to both the accumulated length l_a and the batch size b when l_a is below $2k$:

$$t_d(b, l_a) = \alpha_d \cdot b \cdot l_a + \beta_d \cdot b + \gamma_d \cdot l_a + \delta_d \quad (15)$$

Where $\alpha_d, \beta_d, \gamma_d, \delta_d$ denote as fitting coefficients for decode time. The decode time $t_d(b, l_a)$ can be expressed as the accumulation of the per-token decode time, with dependency to the batch size b , the input length l_i , and the output length l_o :

$$t_d(b, l_i, l_o) = \sum_{k=1}^{l_o} t_d(b, l_i + k) \quad (16)$$

Therefore, three typical SLOs (e2e latency, TTFT, and TPOT), without considering the waiting time, can be expressed as follows:

$$t_{exec}(b, l_i, l_o) = t_p(b, l_i) + t_d(b, l_i, l_o) \quad (17)$$

$$t_{prefill}(b, l_i) = t_p(b, l_i) \quad (18)$$

$$t_{decode}(b, l_i, l_o) = \frac{t_d(b, l_i, l_o)}{l_o} \quad (19)$$

The SLO-aware scheduler employs a request profiler to gather data across a range of b , l_i and l_o . This collected data is instrumental in calculating the precise fitting coefficients: $\alpha_p, \beta_p, \gamma_p, \delta_p, \alpha_d, \beta_d, \gamma_d$, and δ_d . Equations 14 and 15 depict multiple linear regression models that include interaction terms. To determine the coefficients, a set of data with diverse values of l_i , l_o and b is sampled. The objective is to identify the coefficient set that minimizes the mean squared error between the predicted and actual values of t_p and t_d , and coefficient set can be solved using the least squares method.

The remaining inquiries now are: 1) how to ascertain the output length l_o , and 2) how to determine the optimal batch size b ? For the first question, the SLO-aware scheduler gathers data on the task types of incoming requests and offers an optional input variable to allow business users to specify a typical output range or distribution for each task type. If the output range or the distribution is unavailable, the request profiler is equipped with the capability to gather and model the output range and distribution for various task types. This data can subsequently be leveraged to ascertain a probable typical output length, as well as the boundaries for output length. Moreover, established designs, such as S3 [16] and Ref [41], offer precise predictions of output lengths. These can be integrated with the SLO-aware scheduler to enhance the accuracy of latency predictions.

For the second question, as depicted in Equations 17, 18, and 19, the e2e/prefill/decode latencies are influenced by the batch size, which in turn affects the establishment of the priority sequence. Once the priority sequence is determined, the batch size also needs to be updated accordingly restricted by the remaining memory size and the preset maximum batch size. Consequently, the batch size and the priority sequence cannot be optimized separately. The method of determining the optimal batch size b , will be elaborated upon in Sections 4.3 and 4.4.

4.3 Priority Mapping Algorithm

As detailed in Section 4.2, the batch sizes and the priority sequence must be fine-tuned in a coordinated fashion. Consequently, within the priority mapper, the fundamental strategy for optimizing both parameters involves fixing one (either the batch sizes or the priority sequence) while optimizing the other, and then iterating this process to achieve a refined optimization.

According to the definition of G , its upper boundary is achieved if all requests successfully meet their SLOs when the priority sequence is perfectly aligned with the requests' e2e latency order. In this scenario, the latency summation of all requests t is the smallest with the largest achievable n . If not all requests achieve their SLOs when the priority sequence matches the requests' e2e latency, the priority order that maximizes the objective G needs to be searched.

Given a predefined batch size, input request length, and a potential output length, the latency predictor provides a reasonable estimation of the e2e latency, TTFT, and TPOT, excluding any waiting time. Subsequently, with an assumed execution order, the objective function G (as outlined in Equation 2) can be calculated. Modifying the batch sizes and the execution order yields varying values of G . By documenting the maximum value of G , the optimized batch sizes and the corresponding priority sequence can be ascertained.

Strawman Solution: exhaustive search. To determine the priority sequence and the batch size that yields the highest value for the objective G , a straightforward approach involves systematically evaluating every permutation of the request execution order and every possible request batch size combination, calculating the corresponding G for each, and selecting the setting of the batch sizes and the sequence permutation that results in the maximum G . Nevertheless, this method of generating all permutations is associated with a time complexity of $O(N!)$, and finding all batch size combination has a time complexity of $O(2^N)$. The total time complexity of traversing over all possible solution is $O(N! \cdot 2^N)$, which can be computationally infeasible when the number of requests N grows. A scheduler employed in practical scenarios cannot accommodate such an overhead, as the scheduling overhead directly contributes to the e2e request processing latency.

Our Optimized Solution: Simulated Annealing. An effective strategy for addressing the challenge is to employ simulated annealing to explore sequence permutations and request batch sizes that yield the maximum value of G . Simulated annealing, a classic heuristic optimization technique, is renowned for its ability to approximate global optima and has found widespread application across diverse fields [9, 12, 28]. Drawing inspiration from the process of cooling in materials, this algorithm introduces a random perturbations to the current solution, generating a new solution in the process. The new solution is unconditionally accepted if it

Algorithm 1: Simulated-Annealing Priority Mapping

Input: Newly arrived jobs J_{in} with valid predicted latency $J_{in}.predE2E$, $J_{in}.predTTFT$, $J_{in}.predTPOT$, simulated annealing parameters initial temperature T_0 , threshold temperature T_{thres} , iteration times $iter$, temperature decay rate τ

Output: Job priority $job.prio$, $job \in J_{in}$

```

1 procedure priorityMapping
2 Initialization:  $T = T_0$ 
3  $J_{prio} = sorted(J_{in}, key = J_{in}.predE2E)$ 
4  $f = calculateG(J_{prio})$ 
5  $k = 0$ 
6 // check if the sequence with smallest accumulated latency have
all requests meet SLOs
7 if  $meetSLONum(J_{prio}) == len(J_{prio})$  then
8   for  $job \in J_{prio}$  do
9      $job.prio = job.index(J_{prio})$ 
10  return
11 else
12    $f_{init} = calculateG(J_{in})$ 
13   // find the better start point from the sequence with smallest
accumulated latency and the initial sequence
14   if  $f < f_{init}$  then
15      $J_{prio} = J_{in}$ 
16 repeat
17   repeat
18      $k += 1$ 
19     // choose a way to generate a new solution
20      $operation = rand(0, 1, 2)$ 
21     if  $operation == 0$  then
22        $J_{cur} = squeezeLastIter(J_{prio})$ 
23     else if  $operation == 1$  then
24        $J_{cur} = delayNextIter(J_{prio})$ 
25     else
26        $J_{cur} = randSwapping(J_{prio})$ 
27      $f_{new} = calculateG(J_{cur})$ 
28     // determine whether to accept the new solution
29     if  $f_{new} > f$  then
30        $J_{prio} = J_{cur}$ 
31        $f = f_{new}$ 
32     else if  $exp(-\frac{f_{new}-f}{T}) < rand(0, 1)$  then
33        $J_{prio} = J_{cur}$ 
34        $f = f_{new}$ 
35   until  $k == iter$ ;
36    $T * = \tau$ 
37 until  $T < T_{thres}$ ;
38 for  $job \in J_{in}$  do
39    $job.prio = job.index(J_{prio})$ 

```

improves upon the current score. Conversely, even if the new solution is less effective, it may still be accepted with a probability, thus facilitating the escape from local optima.

To incorporate simulated annealing into the priority mapping process, the pseudo code is listed in Algorithm 1. There are two starting solutions: one is the original input sequence order with all request batch sizes set to the maximum; another is generated by aligning the priority sequence according to the e2e latency of the

requests. If the second solution satisfies the SLOs for all requests, the search is terminated, and the optimal solution is deemed to have been discovered. If not, the one starting solution that has higher G is set as the start point, and the simulated annealing algorithm commences its search: the temperature T decays, and at each temperature level the process iterates $iter$ times to approximate the optimal solution. During this phase, the algorithm has three possible ways to create a new sequence: squeezing a request into the last batch iteration if the current request is not in the first iteration and the last iteration batch size has not reach the maximum; delaying a request into the next iteration if the next iteration batch size has not reach the maximum; randomly exchanging the positions of two requests in the priority sequence. If the objective function G for the new sequence exceeds that of the previous sequence, it is adopted. Alternatively, even if the new sequence yields a lower G value, it may still be accepted based on a probability that is dependent on the current temperature T : the higher the temperature, the greater the likelihood of accepting a less optimal solution. The search terminates once the temperature falls below the predefined threshold.

The overall time complexity of the search process is $O(t \cdot iter)$, with t represents the number of steps required for the temperature to decrease to the threshold level. Significantly, the computational time for the simulated annealing search does not scale linearly with the number of requests. Nonetheless, to preserve the effectiveness, it is imperative to adjust the hyperparameters T and $iter$ upwards when the number of requests increases, as this expansion accommodates a broader search space.

Algorithm 2: SLO-Aware Scheduling

Input: Newly arrived jobs J_{in} , remaining memory per instance

$M_0, M_1, \dots, M_{InstNum-1}$, and instances queues

$Q_0, Q_1, Q_{InstNum}$

Output: Jobs to be executed per instance

$J_{out,0}, J_{out,1}, \dots, J_{out,InstNum-1}$ for one iteration

```

1 procedure SLOAwareScheduling
2 Initialization:  $J_{out,0}, J_{out,1}, \dots, J_{out,InstNum-1} = 0, 0, \dots, 0$ 
3 // instance pre-assignment; job latencies are predicted in this stage
4  $J_{in,0}, J_{in,1}, \dots, J_{in,InstNum-1} = InstAssign(J_{in})$ 
5 for  $J_{cur} \in \{J_{in,0}, J_{in,1}, \dots, J_{in,InstNum-1}\}$  do
6   // use the latency prediction and the SLOs to decide priority
7    $J_{cur} = priorityMapping(J_{cur})$ 
8    $J_{cur} = sorted(J_{cur}, key = J_{cur}.priority)$ 
9   // instance assignment
10  for  $job \in J_{cur}$  do
11     $Q_{cur}.push(job)$ 
12 // schedule jobs to execute
13 for  $Q \in \{Q_0, Q_1, \dots, Q_{InstNum}\}$  do
14    $inst = Q.index$ 
15    $J_{out,inst} = ScheduleReq(J_{in}, Q, M_{inst})$ 
16 return  $J_{out,0}, J_{out,1}, \dots, J_{out,InstNum-1}$ 

```

4.4 Scheduling Solution

The application of SLO-aware scheduling can be expanded to encompass multiple instances as needed. Consequently, the scheduling solution encompasses both instance assignment and request

scheduling. The instance assignment plays a crucial role in determining which requests are prioritized during the priority mapping phase. Ideally, the instance assignment and priority mapping should be coordinated to maximize the value of G across all requests. However, this approach encounters the challenge of a search space that expands linearly with the number of requests, which can impede efficiency even when employing simulated annealing for the search process, particularly in scenarios with a high volume of concurrent requests. To address this, the scheduling solution first allocates requests to instances and then performs priority mapping within each instance independently. If LLM instances are hosted on different servers, the priority mapping can be carried out in a distributed fashion, thereby further accelerating the process.

The operational flow of the scheduling solution is as follows: initially, requests latencies are predicted, and distributed to instances in a round-robin fashion. Next, the priority sequence is established through the priority mapping algorithm, as described in Section 4.3. Requests are then enqueued to instance-specific queues and scheduled for execution on LLM instances. The pseudo-code for this scheduling process is provided in Algorithm 2.

Instance Assignment. Following the aforementioned workflow, the logic for assigning request sequences to instances is to first predict the latency of the requests, and then allocate requests in a round-robin way, with each request being assigned to the instance possessing the largest remaining memory, thereby achieving load balancing. When the instance with the largest remaining memory lacks sufficient space to accommodate a new instance, the remaining memory is reset, indicating that a maximum possible number of requests have been allocated and a fresh iteration starts. The equation to estimate the number of tokens with a given memory usage is as follows:

$$token_num(m) = \frac{m \cdot \mu}{\sigma} \quad (20)$$

Where m is the remaining memory, μ denotes the memory utility that is less than 1 due to memory fragmentation, and σ is the memory consumption per token. In actual practice, μ and σ need to be ascertained. μ is adjusted by computing the ratio of the peak memory consumption to the available memory prior to executing any inference and then averaging this ratio. Meanwhile, σ can be derived by dividing the aggregate memory consumption by the total number of tokens processed.

Request Scheduling. The request scheduling is straightforward: once the final priority sequence is determined and each request is assigned to an instance, the requests are then dispatched into the queue associated with their respective instances. A batch of requests is then scheduled for execution from this queue as soon as the corresponding LLM instance is prepared and available.

5 EVALUATION

In this section, we evaluated the SLO-aware scheduler through a series of experiments. Initially, we showcase the overall performance enhancements over varied frameworks (vLLM and LMDe-ploy), models (Qwen2.5-7B and Qwen2.5-32B), and devices (NVIDIA V100 GPUs and NVIDIA A800 GPUs). Next, we delve into the influence exerted by individual features. Finally, we test the scalability across instances.

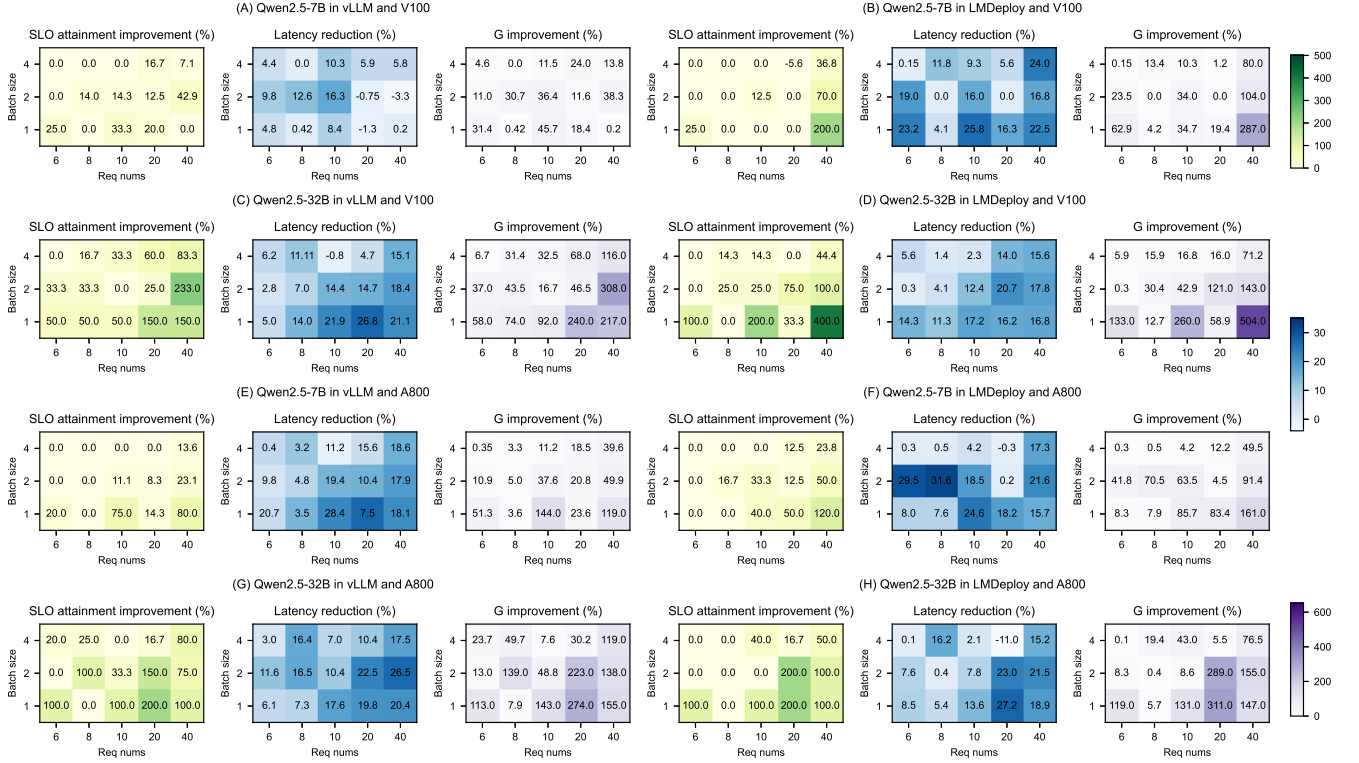


Figure 7: Overall performance with (A) Qwen2.5-7B in vLLM in V100, (B) Qwen2.5-7B in LMdeploy in V100, (C) Qwen2.5-32B in vLLM in V100, (D) Qwen2.5-32B in LMdeploy in V100, (E) Qwen2.5-7B in vLLM in A800, (F) Qwen2.5-7B in LMdeploy in A800, (G) Qwen2.5-32B in vLLM in A800, (H) Qwen2.5-32B in LMdeploy in A800, evaluated in a mixed dataset of ShareGPT_Vicuna_unfiltered dataset and Python-Code-23k-ShareGPT dataset with varying request numbers and maximum batch sizes.

5.1 Experimental Settings

Testbed. The evaluation utilizes 2 NVIDIA V100 32GB GPUs or 1 NVIDIA A800 80GB GPUs. The NVIDIA V100 GPUs are connected over PCIe 3.0 x16, equipped with 516GB host memory and 2 Intel Xeon 6132 CPUs. The NVIDIA A800 GPU is equipped with 1546GB and 2 Intel Xeon 6240 CPUs. The scalability tests were deployed to up to 8 V100 GPUs in the same server.

LLM models. The LLM models utilized in the experiment are Qwen2.5-7B and Qwen2.5-32B[38], which have been widely used in industry and academia. For Qwen2.5-7B, one instance utilizes 2 NVIDIA V100 GPUs, and FP16 precision is used during the inference. Experiments with Qwen2.5-7B in 1 A800, Qwen2.5-32B in 4 V100s, and Qwen2.5-32B in 1 A800 were also delivered.

Workloads. The dataset employed for this experimental study is a composite of the ShareGPT_Vicuna_unfiltered dataset [15] and the Python-Code-23k-ShareGPT dataset [14], both of which are parts of the ShareGPT collection [32]. However, they cater to distinct task domains: the former is tailored for chatbot interactions, while the latter is designed for code generation. In current experiment, the combined dataset is constructed by evenly selecting requests from each source, with each request being tagged according to its respective task type. For the requests sourced from the dataset ShareGPT_Vicuna_unfiltered, the evaluation metrics are

TTFT and TPOT, whereas for the requests from Python-Code-23k-ShareGPT, the focus is on measuring the e2e latency because a code is useful only when completed. Request lengths in both datasets are restricted to under 2k for the latency predictor’s validation.

SLOs. The SLOs reflect user experiment expectations for a certain task. Historical research [26] suggests establishing the e2e latency SLO at 10× the duration of processing an individual request. Based on profiling data with Qwen2.5-7B in 2 V100 GPUs, given a request of the Python-Code-23k dataset with an average length, the time required to process it is 3s. Consequently, the e2e latency SLO is set at 30s. For interactive tasks like a chatbot, the TTFT is defined as 10s, and the TPOT is 50 milliseconds. In practical applications, SLOs influence model deployment and the maximum allowable batch sizes for LLM services. Nevertheless, to demonstrate the impact of the SLO-aware scheduler, we maintain the SLOs for all experiments.

Evaluation metrics. The metrics for evaluating the experiments encompass the objective function G , as delineated in Equation 2. Furthermore, to explore the impact of SLO-aware scheduling, we

have also presented the SLO attainment rates and the average latency metrics. In addition, we have quantified the overhead associated with implementing the SLO-aware scheduling mechanism.

Baselines. The simulated-annealing SLO-aware scheduler operates as a dedicated scheduling component, necessitating its integration within a LLM service. For the experimental evaluation, we have adopted vLLM v0.6.0, a state-of-the-art LLM serving system renowned for its support of PageAttention and continuous batching, as our baseline platform. The simulated-annealing SLO-aware scheduler is implemented atop this vLLM system. Besides, we also used LMDeploy v0.7.2 to validate the effectiveness in varied framework.

Moreover, to assess the performance and efficiency of the simulated annealing-based SLO-aware scheduler, we have also conducted a comparative analysis with an exhaustive search-based SLO-aware scheduler integrated within the vLLM framework. Due to the inefficient searching of exhaustive searching, the comparison was confined to small batch sizes and a rather limited number of requests.

Implementations. The experiment script is implemented as 1.7k lines of *Python* code. The default values for the hyperparameters are set as follows: T_0 to 500, T_{thres} to 20, $iter$ to 100, and τ to 0.95 unless otherwise specified. As suggested by official instructions, the memory utilizing efficiency is set to be 0.9 for vLLM.

Workflows. The procedure of the experiments was conducted as follows: Initially, following the deployment of a LLM instance, a series of profiling rounds were carried out to gather data on the e2e, prefill, and decode latencies across a range of batch sizes and request lengths. For this study, profiling was performed with the maximum batch size varying from 1 to 32, and the request lengths ranging from 100 tokens to 8,000 tokens. The fitting coefficients are stored to be used for later latency prediction.

Next, the mixed dataset was created by randomly sampling an equal number of requests from two datasets: Python-Code-23k-ShareGPT and ShareGPT_Vicuna_unfiltered dataset. Following this, the dataset was shuffled to randomize the order of the entries. To ensure a fair comparison across the baseline, for the exhaustive-search SLO-aware scheduling and the simulated-annealing SLO-aware scheduling, the same random seed was utilized. However, the seed was modified when applying different experimental settings to prevent the reuse of the same subset of the dataset.

The output length predictor tracked the actual lengths of the outputs once a request's response was produced, and dynamically modeled this data using a Gaussian distribution. To estimate the output length for a new request, the predictor generates a random integer based on this fitted distribution.

When the SLO-aware feature was disabled, requests were dispatched to the baseline service continuously, allowing the service to determine the actual batch size for each iteration. Conversely, when SLO-aware scheduling was activated, requests were submitted to the baseline service in a predetermined order and batch size: requests within the same batch were sent concurrently, whereas requests in separate batches were separated by a brief interval of 0.1ms to prevent them from being combined into a single batch.

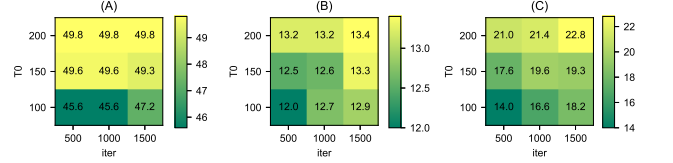


Figure 8: Improvement of G with varying T_0 and $iter$ in simulated annealing-based priority mapping. (A) involves 10 requests with a maximum batch size of 1, (B) includes 20 requests with a maximum batch size of 2, and (C) features 40 requests with a maximum batch size of 4.

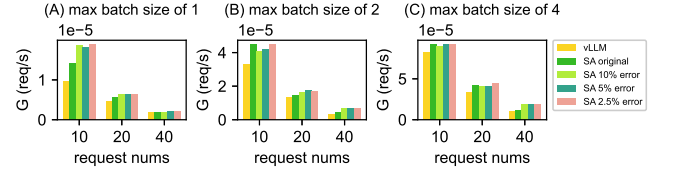


Figure 9: Impact of output length prediction on the enhancement in simulated annealing SLO-aware scheduler under the maximum batch sizes of (A) 1, (B) 2, and (C) 4.

5.2 Overall Performance

We compared the SLO attainment, the average latency, and their ratio (denoted as G) across varying request numbers and batch sizes. This evaluation was performed using the simulated-annealing SLO-aware scheduler (SA), an exhaustive-search counterpart, and vLLM which lacks SLO-awareness. Figure 7 presents the experimental outcomes for (A) Qwen2.5-7B in vLLM in V100, (B) Qwen2.5-7B in LMdeploy in V100, (C) Qwen2.5-32B in vLLM in V100, (D) Qwen2.5-32B in LMdeploy in V100, (E) Qwen2.5-7B in vLLM in A800, (F) Qwen2.5-7B in LMdeploy in A800, (G) Qwen2.5-32B in vLLM in A800, (H) Qwen2.5-32B in LMdeploy in A800.

Owing to the variability in request sequences, the enhancement of the metric G spans a range from 0 to 504%. Occasionally, the initial request execution order was sufficiently efficient with minimal waiting times, leading to only modest improvements from the simulated annealing process (as observed in the case with 40 requests and a batch size of 1 in Figure 7 (A)); or even no improvement (as seen with 8 requests and a batch size of 4 in Figure 7 (A)), as the result of variations in execution time. However, compared with vLLM and LMdeploy, in most cases the simulated-annealing SLO-aware scheduler shows prominent improvement. Among results with varied experiment settings showed growths up to 5 $\times$ in SLO attainment (with 40 requests and the maximum batch size of 1 in Figure 7(D)) and up to 31.6% (with 8 requests and the maximum batch size of 2 in Figure 7(F)) in average latency. A case with strict SLOs and a large request number (as shown in Figure 7(G)-(H)) produced a worse baseline and a larger search space, hence showed a more significant improvement in SLO attainment. On the contrary, the average latency improvement depended on more the baseline sequence randomness than the model or framework performance. Besides, it can also be seen that sometimes the SLO attainment (seen in Figure 7 (B)) or the latency (seen in Figure 7 (A), (C), (F) and (H)) is sacrificed to achieve an overall higher G .

req num	SA				Exhaustive search			
	SLO attainment	Latency (s)	G ($\times 10^{-3}$)	Overhead (s)	SLO attainment	Latency (s)	G ($\times 10^{-3}$)	Overhead (s)
4	1	10.29	9.72	0.00023	1	10.18	9.82	0.0012
6	0.83	11.44	7.26	0.00032	0.83	12.00	6.92	0.039
8	0.875	14.48	6.04	0.00043	0.875	14.50	6.03	88.2
10	0.4	27.61	1.45	0.00048	0.4	28.38	1.41	287

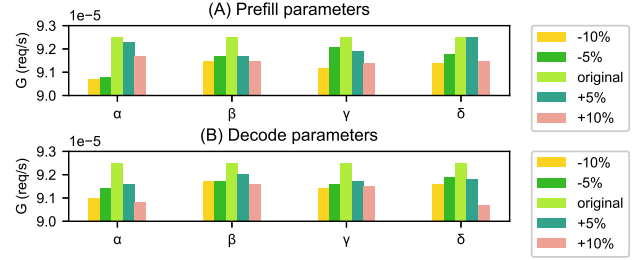
Table 1: Comparison of simulated annealing-based priority mapping and exhaustive search.

Furthermore, the simulate annealing-based priority mapping solution demonstrated performance with a maximum degradation of just 1.0% across the tested examples, compared to the exhaustive counterpart. We further compared the execution overhead of two priority algorithms with the maximum batch size of 1 and the request number across 4 to 10 and showed the results in Table ?? . Results for request numbers exceeding 10 were not recorded, due to prohibitively long computing time from exhaustive search. As request numbers increase, the exhaustive search solution's execution time exhibits exponential growth, whereas that of the simulated annealing method demonstrates only a slight increase, which is attributed to the overhead of additional request management. This validates the efficiency of our simulated-annealing solution.

We conducted further experiments to investigate the influence of the initial temperature T_0 and the number of iterations $iter$ on the improvement of G within our simulated-annealing solution. The outcomes are depicted in Figure 8. Overall, the results indicate that under a specific temperature, increasing the initial temperature T_0 yields more significant improvement in G compared to increasing the iteration time $iter$. For an instance, in the case with 10 request and a maximum batch size of 1 (as illustrated in Figure 8 (A)), the improvement of G has grown from 45.6% to 49.8% when the initial T_0 was elevated from 100 to 200, whereas increasing $iter$ shows an less substantial improvement in G (i.e., from 45.6% to 47.2%). Examples with different batch sizes and request numbers depicted in Figure 8 (B) and (C) also shows this tendency. This is because rising the initial temperature T_0 not only increments the number of iterations but also heightens the probability of accepting suboptimal solutions, thereby providing more opportunities to escape local optima. However, it is not always beneficial to have excessively high values for T_0 or $iter$. While a higher T_0 increases the probability of accepting suboptimal solutions, it also necessitates a correspondingly higher $iter$ to explore more effectively for superior solutions. Moreover, a substantial T_0 or $iter$ can result in decreased efficiency, as the computational time is directly proportional to these parameters. In practice, it is advisable to fine-tune T_0 and $iter$ in alignment with the size of the actual search space to optimize performance.

5.3 Impact of Output Length Prediction

We have also explored the effects of output length prediction on our system. Currently, the SLO-aware scheduler utilizes profiling data from tasks to dynamically update the mean and the standard deviation of tasks' actual outputs, which in turn are employed to predict the output lengths of incoming requests. The experimental setup was conducted as follows: rather than relying on profiling data to predict output lengths, we employed actual output lengths

**Figure 10: Impact of variations on fitting parameters of the latency predictor with 10 requests and the maximum batch size of 4.**

parameter	α	β	γ	δ
for prefill	0.1	5.7	0.01	43.67
for decode	0.0002	0.275	0.00088	15.85

Table 2: Fitting parameters used in the latency predictor.

with variations of 2.5%, 5%, and 10% to simulate the use of an output length predictor with different levels of accuracy. Figure 9 illustrates the impact of the accuracy of output length prediction on the performance enhancement of our SLO-aware scheduler, across different maximum batch sizes: 1 (as shown in Figure 9 (A)), 2 (as shown in Figure 9 (B)), and 4 (as depicted in Figure 9 (C)).

A discernible trend emerges, indicating that the greater the accuracy of the output length prediction, the more enhancement compared to the current solution, though the inherent randomness of the simulated annealing process still introduces slight variations in the actual outcomes. For example, in the scenario with 40 requests and a maximum batch size of 4 (as depicted in Figure 9 (C)), employing a single output length predictor with an error margin of less than 2.5% resulted in a 65% increase in G over the original SLO-aware scheduler, and an 84% enhancement in G when compared to the baseline. While developing a highly accurate output length predictor is beyond the scope of this work, the findings nonetheless showcased the benefits of incorporating an accurate output length predictor with the SLO-aware scheduler in real-world applications. These results also motivated us to enhance the predictor's effectiveness in future research endeavors.

5.4 Impact of Latency Prediction

Adhering to the methodology illustrated in Section 4.2, the fitting parameters derived from the profiling data are presented in Table 2. Nonetheless, in practical scenarios, factors such as inadequate sampling may lead to variations in these parameters, potentially resulting in the usage of suboptimal fitting parameters. To investigate the effects of such variations, we conducted an experiment using a sample scenario with 10 requests and a maximum batch size of 4. The results of this experiment are depicted in Figure 10.

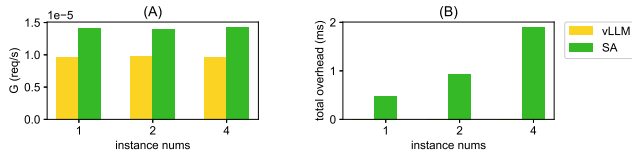


Figure 11: The enhancement of G (A) and the overall overhead (B) when applied across multiple instances on the same machine.

Overall, there is a correlation between the degree of deviation from the optimal parameters and the severity of degradation in G , with the most significant decline reaching up to 1.9%. Furthermore, variations in α have been identified as the most impactful, as it directly affects the accuracy of estimating the influence of batch size and processing lengths. Therefore, it is recommended to collect a sufficient amount of data for parameter fitting to mitigate the effects of variations in the fitting parameters.

5.5 Scalability Tests

We have expanded our experiments to encompass multiple instances. Figure 11 illustrates the impact of the SLO-aware scheduler as the number of instances ranges from 1 to 4. In this setup, each instance was allocated to 2 V100 GPUs, with all instances collocated on a single server. To mitigate the effects of randomness, 10 requests were dispatched to a single instance, and these requests were then replicated across all instances, with the number of copies matching the number of instances. The figure clearly demonstrates that the effectiveness of the SLO-aware scheduler is sustained when scheduling is performed within each instance. There is a linear increase in the time taken for scheduling, which rises to 0.93 ms for 2 instances and 1.91 ms for 4 instances. This escalation is due to the sequential execution of multiple instances on the same server, which can be accelerated by parallelism in the future.

6 CONCLUSION

In this paper, we introduced a novel priority mapping method and invented the first SLO-aware scheduler designed for scenarios where LLM inference requests exhibit varying SLOs. With the goal of optimizing system SLO compliance while maintaining the lowest possible average latency, we computed the priority of each request and determine the optimal batch size for execution based on task-specific SLOs, input lengths, and output lengths prediction. Through the simulated annealing-based SLO-aware scheduling, we achieved comparable effectiveness to the exhaustive search-based counterpart while incurring an overhead of merely 1ms. We further evaluated the effectiveness of our system against the cutting-edge schedulers vLLM and LMDeploy using the mixed dataset of Python-Code-23k-ShareGPT and ShareGPT_Vicuna_unfiltered dataset, demonstrating that our solution can boost the SLO attainment by up to 5× and reduce the average latency by up to 31.6%, respectively.

7 ACKNOWLEDGMENT

We acknowledge ChatGLM for AI-assisted proofreading during the writing of the whole paper.

REFERENCES

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *arXiv preprint arXiv:2403.02310* (2024).
- [2] Open AI. 2022. "Introducing ChatGPT". <https://openai.com/blog/chatgpt>
- [3] Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. 2024. LeanContext: Cost-efficient domain-specific question answering using LLMs. *Natural Language Processing Journal* 7 (2024), 100065. <https://doi.org/10.1016/j.nlp.2024.100065>
- [4] Brian Chen, Nafis Mustakin, Alvin Hoang, Sakib Fuad, and Daniel Wong. 2023. VSCuda: LLM based CUDA extension for Visual Studio Code. In *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis* (Denver, CO, USA) (SC-W '23). Association for Computing Machinery, New York, NY, USA, 11–17. <https://doi.org/10.1145/3624062.3624064>
- [5] Ke Cheng, Wen Hu, Zhi Wang, Hongen Peng, Jianguo Li, and Sheng Zhang. 2024. Slice-Level Scheduling for High Throughput and Load Balanced LLM Serving. *arXiv:2406.13511 [cs.DC]* <https://arxiv.org/abs/2406.13511>
- [6] Gobinda Chowdhury and Sudatta Chowdhury. 0. AI- and LLM-driven search tools: A paradigm shift in information access for education and research. *Journal of Information Science* 0, 0 (0), 01655515241284046. <https://doi.org/10.1177/01655515241284046>
- [7] LMDeploy Contributors. 2023. LMDeploy: A Toolkit for Compressing, Deploying, and Serving LLM. <https://github.com/InternLM/lmdeploy>.
- [8] John Dagdelen, Alexander Dunn, Sanghoon Lee, Nicholas Walker, Andrew Rosen, Gerbrand Ceder, Kristin Persson, and Anubhav Jain. 2024. Structured information extraction from scientific text with large language models. *Nature Communications* 15 (02 2024). <https://doi.org/10.1038/s41467-024-45563-x>
- [9] Daniel Delahaye, Supatcha Chaimatanan, and Marcel Mongeau. 2018. Simulated annealing: From basics to applications. In *Handbook of metaheuristics*. Springer, 1–35.
- [10] Damiano Duranti, Paolo Giorgini, Andrea Mazzullo, Marco Robol, and Marco Roveri. 2025. LLM-Driven Knowledge Extraction in Temporal and Description Logics. In *Knowledge Engineering and Knowledge Management*, Mehwish Alam, Marco Rospochoer, Marieke van Erp, Laura Hollink, and Genet Asefa Gesese (Eds.). Springer Nature Switzerland, Cham, 190–208.
- [11] S. Egger, T. Hossfeld, R. Schatz, and M. Fiedler. 2012. Waiting times in quality of experience for web based services. In *2012 Fourth International Workshop on Quality of Multimedia Experience*. 86–96. <https://doi.org/10.1109/QoMEX.2012.6263888>
- [12] Bruce Hajek. 1985. A tutorial survey of theory and applications of simulated annealing. In *1985 24th IEEE Conference on Decision and Control*. IEEE, 755–760.
- [13] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. [n.d.]. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Eighth Conference on Machine Learning and Systems*.
- [14] HuggingFace. 2025. Python-Code-23k-ShareGPT. <https://huggingface.co/datasets/ajibawa-2023/Python-Code-23k-ShareGPT>
- [15] HuggingFace. 2025. ShareGPT_Vicuna_unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered
- [16] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S3: increasing GPU utilization during generative inference for higher throughput. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 791, 13 pages.
- [17] Somayeh Koohborfardhaghghi, Gert De Geyter, and Evan Kaliner. 2024. Unlocking the Power of LLM-Based Question Answering Systems: Enhancing Reasoning, Insight, and Automation with Knowledge Graphs. In *Intelligent Systems Design and Applications*, Ajith Abraham, Anu Bajaj, and Thomas Hanne (Eds.). Springer Nature Switzerland, Cham, 156–171.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [19] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, Shen Li, Zhigang Ji, Tao Xie, Yong Li, and Wei Lin. 2024. Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv:2401.02669 [cs.DC]* <https://arxiv.org/abs/2401.02669>
- [20] Lin Liu, Jiajun Meng, and Yongliang Yang. 2024. LLM technologies and information search. *Journal of Economy and Technology* 2 (2024), 269–277. <https://doi.org/10.1016/j.ject.2024.08.007>

- [21] Yixin Liu, Kejian Shi, Katherine S He, Longtian Ye, Alexander R. Fabbri, Pengfei Liu, Dragomir Radev, and Arman Cohan. 2024. On Learning to Summarize with Large Language Models as References. *arXiv:2305.14239* [cs.CL] <https://arxiv.org/abs/2305.14239>
- [22] Meta. 2024. "Introducing Llama 3.1: Our most capable models to date". <https://ai.meta.com/blog/meta-llama-3-1/>
- [23] Nvidia. 2024. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>
- [24] Nvidia. 2024. Triton Inference Server. <https://github.com/triton-inference-server/server>
- [25] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ñigo Gouri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. <https://doi.org/10.1109/ISCA59077.2024.00019>
- [26] George Prekas, Marios Kogias, and Edouard Bugnion. 2017. ZygOS: Achieving Low Tail Latency for Microsecond-scale Networked Tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles* (Shanghai, China) (SOSP '17). Association for Computing Machinery, New York, NY, USA, 325–341. <https://doi.org/10.1145/3132747.3132780>
- [27] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuyong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. *arXiv:2412.15115* [cs.CL] <https://arxiv.org/abs/2412.15115>
- [28] Nazmul Siddique and Hojjat Adeli. 2016. Simulated annealing, its variants and engineering applications. *International Journal on Artificial Intelligence Tools* 25, 06 (2016), 1630001.
- [29] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumix: Dynamic Scheduling for Large Language Model Serving. *arXiv:2406.03243* [cs.AR] <https://arxiv.org/abs/2406.03243>
- [30] Koray Kavukcuoglu, Sundar Pichai, Demis Hassabis. 2024. "Introducing Gemini 2.0: our new AI model for the agentic era". <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/#ceo-message>
- [31] Ze Tang, Jidong Ge, Shangqing Liu, Tingwei Zhu, Tongtong Xu, Liguang Huang, and Bin Luo. 2023. Domain Adaptive Code Completion via Language Models and Decoupled Domain Databases. *arXiv:2308.09313* [cs.SE] <https://arxiv.org/abs/2308.09313>
- [32] ShareGPT teams. 2023. <https://sharegpt.com/>
- [33] Dave Van Veen, Cara Van Uden, Louis Blankemeier, Jean-Benoit Delbrouck, Asad Aali, Christian Bluethgen, Anuj Pareek, Malgorzata Polacin, Eduardo Pontes Reis, Anna Seehofnerova, et al. 2023. Clinical text summarization: adapting large language models can outperform human experts. *Research Square* (2023).
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [35] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Rong Gu, Nguyen Cam-Tu, Chen Tian, and Sheng Zhong. 2024. Revisiting SLO and Goodput Metrics in LLM Serving. *arXiv:2410.14257* [cs.LG] <https://arxiv.org/abs/2410.14257>
- [36] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. *arXiv:2404.09526* [cs.DC] <https://arxiv.org/abs/2404.09526>
- [37] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Distributed Inference Serving for Large Language Models. *arXiv:2305.05920* [cs.LG] <https://arxiv.org/abs/2305.05920>
- [38] An Yang, Baosong Yang, Beichen Zhang, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yyu>
- [40] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. *arXiv:2504.20068* [cs.DC] <https://arxiv.org/abs/2504.20068>
- [41] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. Response length perception and sequence scheduling: an LLM-empowered LLM inference pipeline. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2859, 14 pages.
- [42] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. *arXiv:2401.09670* [cs.DC] <https://arxiv.org/abs/2401.09670>

A EXPERIMENT DETAILED RESULTS

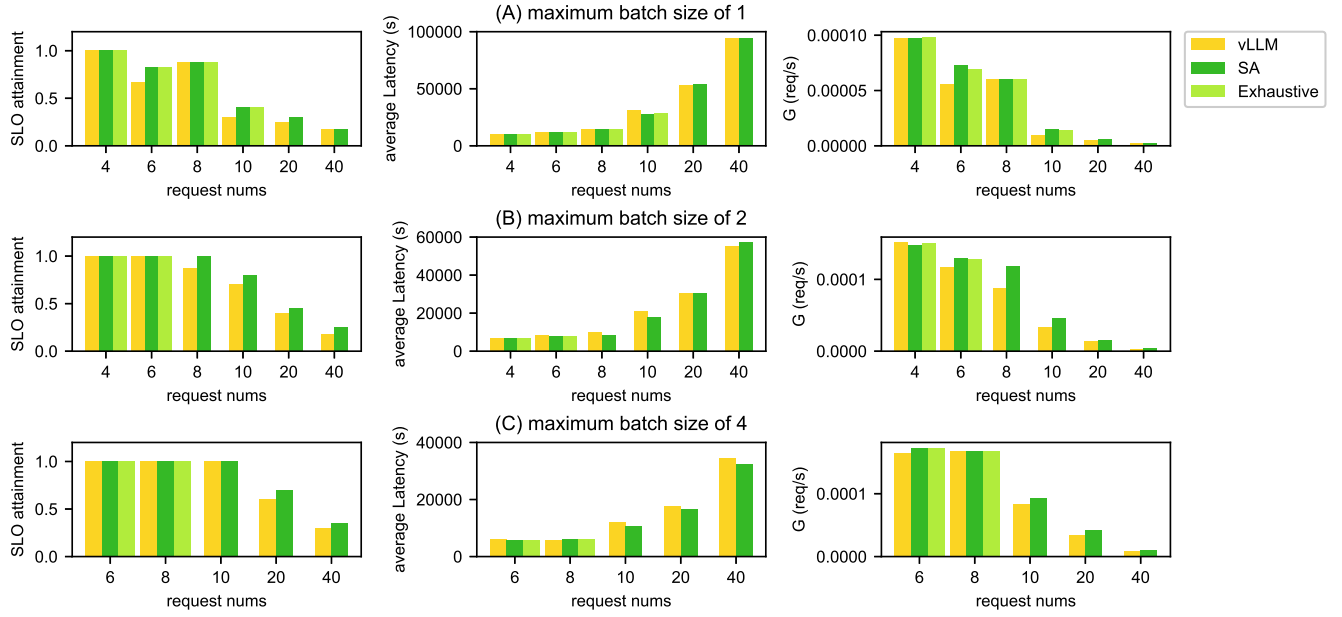


Figure 12: Qwen2.5-7B at 2 Nvidia V100 GPUs in vLLM

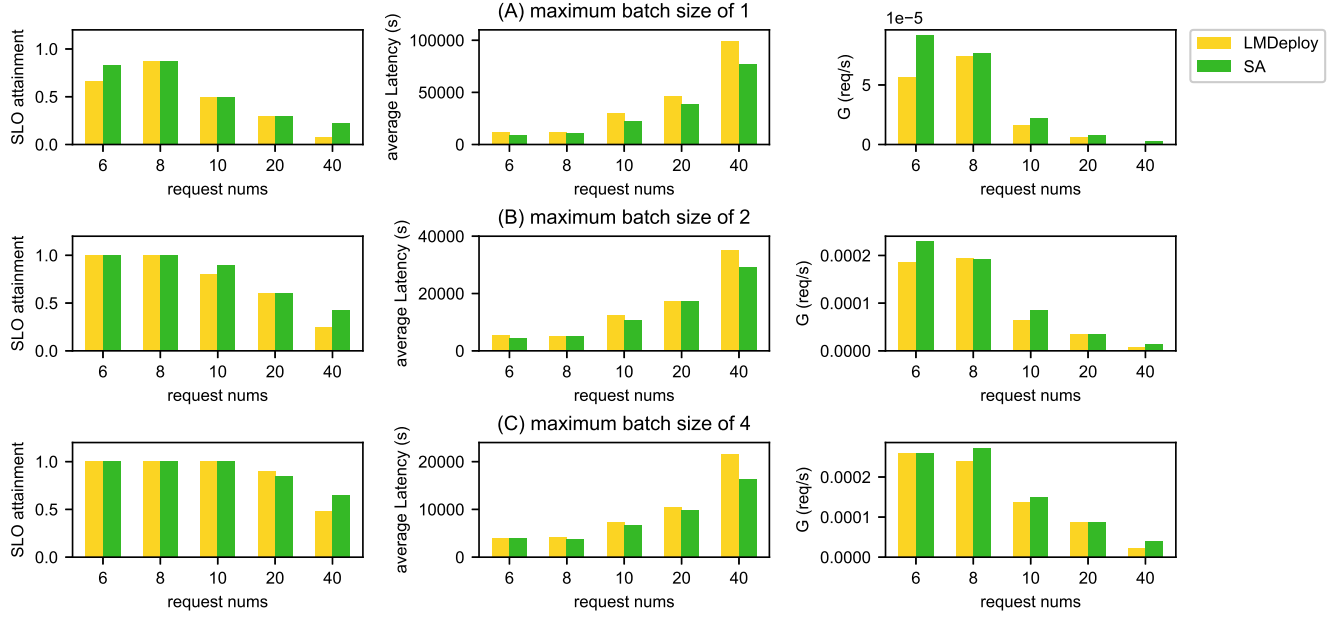


Figure 13: Qwen2.5-7B at 2 Nvidia V100 GPUs in LMDeploy

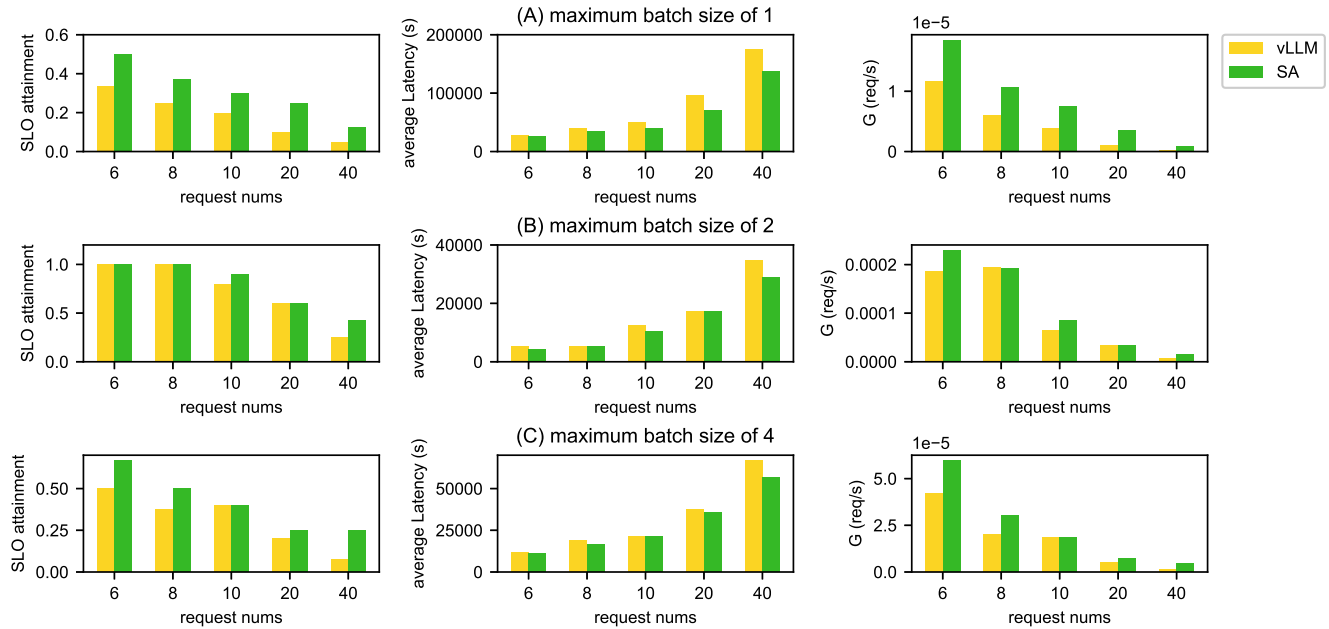


Figure 14: Qwen2.5-32B-Instruct at 4 Nvidia V100 GPUs in vLLM

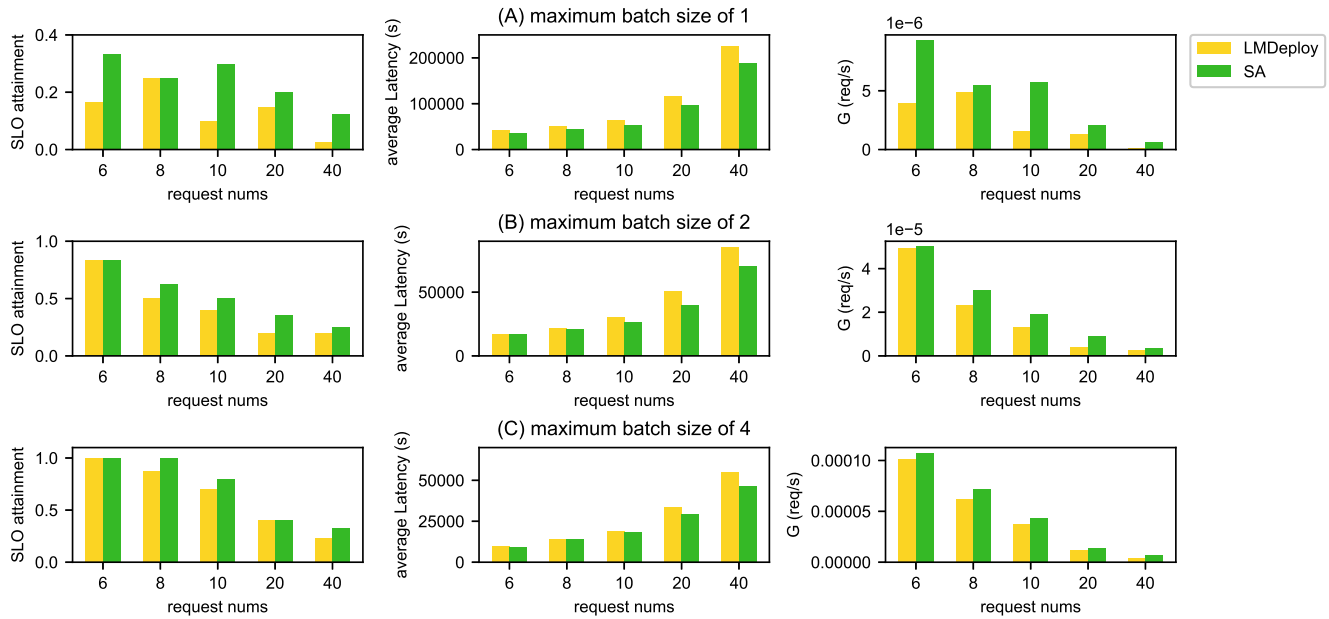


Figure 15: Qwen2.5-32B-Instruct at 4 Nvidia V100 GPUs in LMDeploy

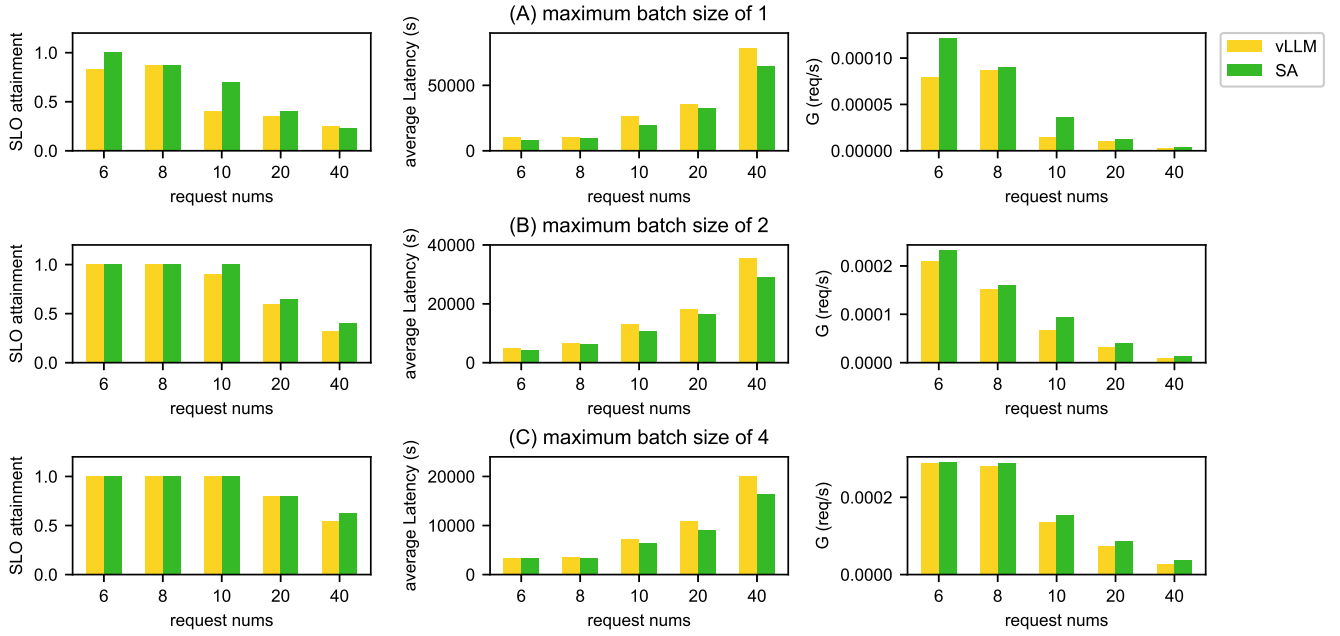


Figure 16: Qwen2.5-7B at 1 Nvidia A800 GPUs in vLLM

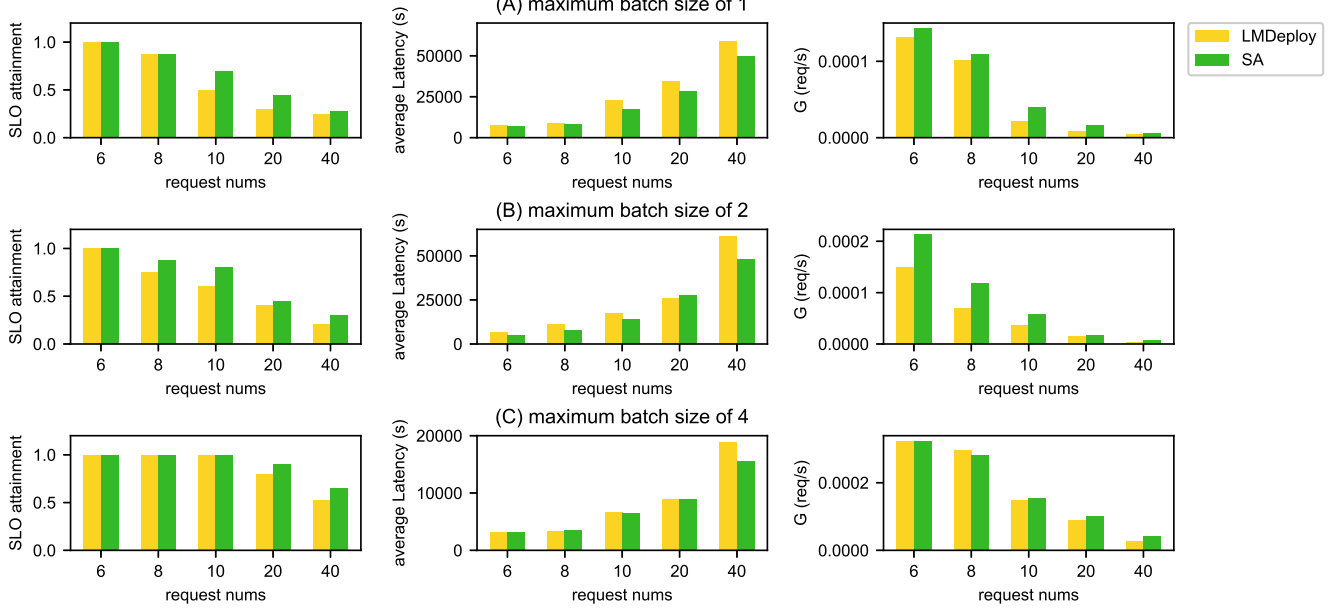


Figure 17: Qwen2.5-7B at 1 Nvidia A800 GPUs in LMDeploy

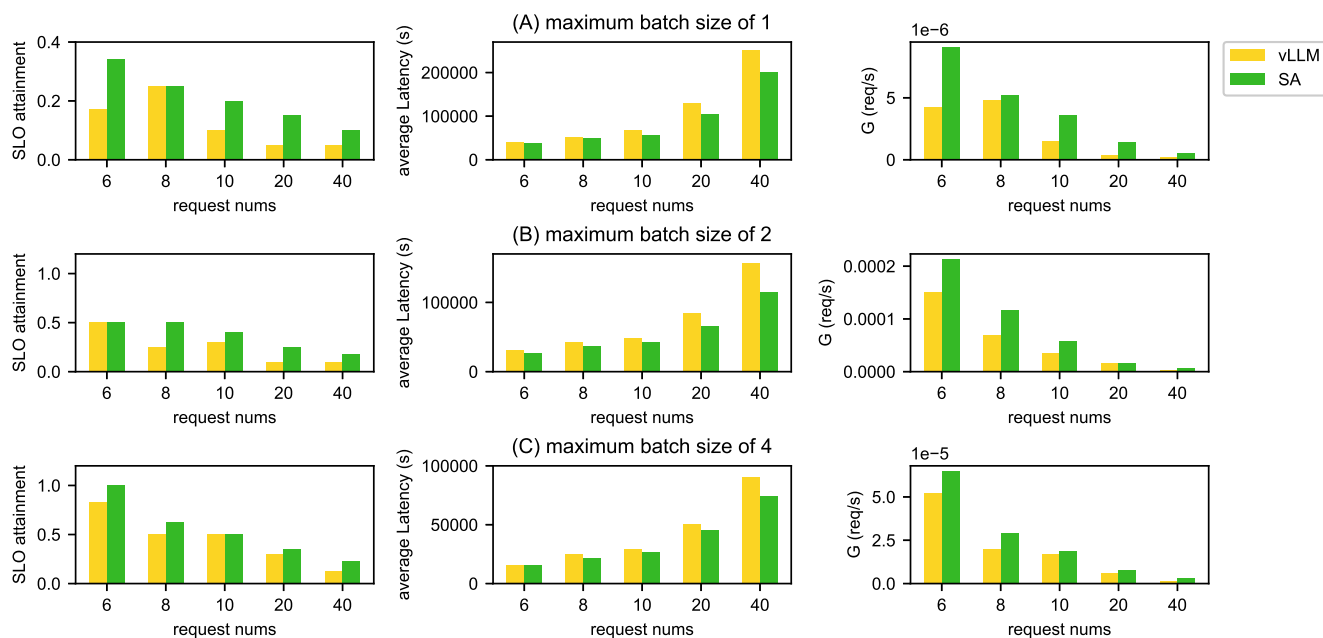


Figure 18: Qwen2.5-32B-Instruct at 1 Nvidia A800 GPUs in vLLM

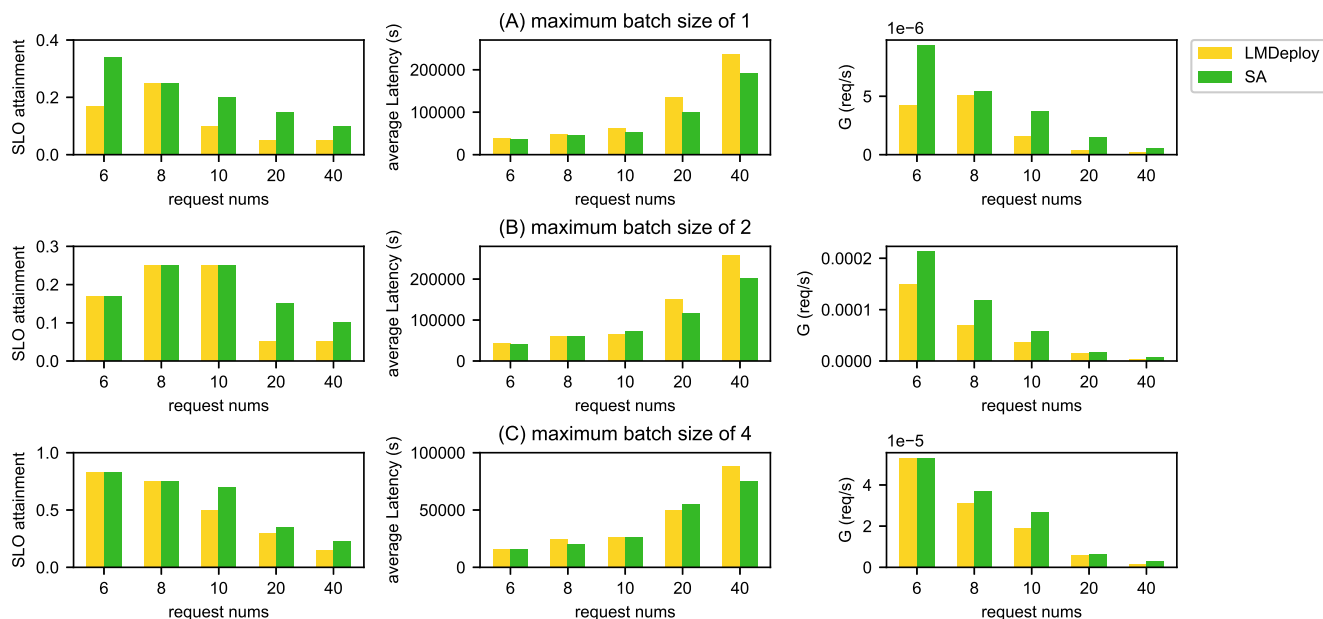


Figure 19: Qwen2.5-32B-Instruct at 1 Nvidia A800 GPUs in LMDeploy